

```
In [1]: import pandas as pd
import numpy as np
import scipy

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np, scipy]:
    print(i.__name__, i.__version__)
```

```
pandas 2.1.4
numpy 1.26.4
scipy 1.11.4
```

2-1 확률 개론

```
In [2]: Image('26.png')
```

Out[2]: **0. 데이터셋 소개**

Titanic

[Titanic](#) 탑승객의 생존 유무를 담은 데이터셋입니다.

Data	Dictionary	Key
PassengerId	Passenger Id	Index
Survived	Survival	0 = No, 1 = Yes
Pclass	Ticket class	1 = 1st, 2 = 2nd, 3 = 3rd
Name	Name	
Sex	Sex	
Age	Age in years	
Sibsp	# of siblings / spouses aboard the Titanic	
Parch	# of parents / children aboard the Titanic	
Ticket	Ticket number	
Fare	Passenger fare	
Cabin	Cabin number	
Embarked	Port of Embarkation	C = Cherbourg, Q = Queenstown, S = Southampton

다양한 형식의 데이터를 가지고 있으며, 여러 가지 아이디어를 생각하고 시도할 만한 요소가 많게끔 기획된 데이터셋입니다.

```
In [2]: df_titanic = pd.read_csv('data/titanic.csv', index_col='PassengerId')
df_titanic.head()
```

Out[2]:

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

In [3]: # Embarked의 결측치는 최빈값으로 대체합니다.

```
df_titanic['Embarked'] = df_titanic['Embarked'].fillna(df_titanic['Embarked'].mode().iloc[0])
```

In [3]: Image('27.png')

Out[3]: **확률의 기본 요소**

1. 시행(Trial): 임의의 결과를 가져오는 어떤 과정

ex) 주사위를 한 번 던지는 행위

2. 결과(Outcome): 시행을 통해 나오게 된 것

ex) 주사위를 던져서 6이 나왔습니다.

3. 표본 공간(Sample Space): 시행에서 얻을 수 있는 가능한 모든 결과의 집합

ex) 주사위 던지기의 표본 공간 = {1, 2, 3, 4, 5, 6}

4. 사건 (Event): 표본 공간의 부분 집합으로, 하나 이상 결과를 포함합니다.

ex) 짝수가 나오는 사건 = {2, 4, 6}

확률

어떤 사건이 일어날 가능성을 0에서 1사이의 수로 나타낸 것입니다.

$$P(A) = \frac{\text{사건 A가 발생한 수}}{\text{전체 시행의 수}}$$

확률의 연산

$P(A \cap B)$, $P(A, B)$: 사건 A와 B가 동시에 일어날 확률

$P(A \cup B)$: 사건 A 또는 B가 일어난 확률: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

$P(A^c)$: 사건 A가 일어나지 않을 확률

$P(A|B)$: 사건 B가 일어날 때, A가 일어날 확률: $P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{\text{사건 } A, B \text{가 동시에 일어난 수}}{\text{사건 } B \text{가 일어난 수}}$

독립 사건

사건 A, B가 독립이라는 것은 하나의 사건이 다른 사건에 아무런 영향을 미치지 않는 것을 의미합니다.

A, B가 독립이라면 아래 식이 성립합니다.

$$P(A|B) = P(A)$$

$$P(B|A) = P(B)$$

$$P(A, B) = P(A)P(B)$$

In [4]: `Image('28.png')`

Out[4]: 조건부 독립

특정 조건이 주어졌을 때, 두 사건이 독립이 되는 것을 의미합니다.

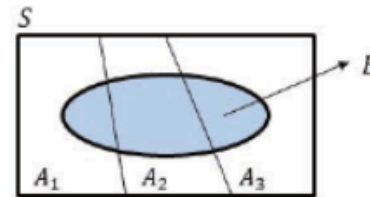
C 조건이 주어졌을 때, A와 B가 서로 독립이 된다면, 아래 식이 성립합니다.

$$P(A|B, C) = P(A|C)$$

$$P(B|A, C) = P(B|C)$$

$$P(A, B|C) = P(A|C)P(B|C)$$

전확률의 법칙(Law of Total Probability)



$$P(B) = P(A_1, B) + P(A_2, B) + P(A_3, B) = \sum_{i=1}^3 P(A_i, B)$$

베이즈 정리(Bayes' Theorem)

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- 전확률의 법칙을 적용해봅시다.

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{P(B)} = \frac{P(B|A_i)P(A_i)}{\sum_{j=1}^n P(B|A_j)P(A_j)} = \frac{P(B|A_i)P(A_i)}{\sum_{j=1}^n P(B|A_j)P(A_j)}$$

베이즈 정리의 요소

베이즈 정리를 이용하면, 사건이 발생하기 전의 확률과 사건의 특성에 대해 확률의 수치를

베이즈 정리를 이용하면, A 사건이 발생할 가능성을 B 사건을 통해서 구해낼 수 있습니다.

이 관점에서 베이즈 정리의 요소들을 아래와 같이 구분할 수 있습니다.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- 사전 확률(Prior): B 사건이 발생하기 전에 알려진 확률입니다. 경험을 토대로 알고 있는 A가 일어날 확률입니다. → $P(A)$
- 사후 확률(Posterior): B 사건이 발생한 후에 조건부로 계산되는 확률입니다. B 사건이 발생한 후에 알게 되는 A가 일어날 확률입니다. → $P(A|B)$
- 우도(likelihood): A 사건이 발생했을 때, B 사건이 일어났을 확률입니다. 또는 A 사건이 발생한다면, B 사건이 일어날 확률입니다. → $P(B|A)$
- 증거(Evidence): B 사건이 발생할 확률. B 사건이 지니는 A 사건에 대한 영향력을 나타냅니다. → $P(B)$

[Ex.1]

df_titanic에서 P(Survived=True) 일 확률을 구해봅시다.

```
In [4]: num_event = len(df_titanic) # 모든 경우의 수
# Survived=True 사건의 수
num_survived = len(df_titanic.loc[df_titanic['Survived'] == True])
p_survived = num_survived / num_event
p_survived
```

Out[4]: 0.3838383838383838

```
In [5]: # Survived가 True인 경우는 아래와 같이 바꿀 수 있습니다.
# boolean 변수 True은 정수로는 1을 의미합니다.
num_survived = (df_titanic['Survived'] == True).sum()
print("num_survived(1):", num_survived)
# 이를 더 줄인다면,
num_survived = df_titanic['Survived'].sum()
print("num_survived(2):", num_survived)
p_survived = num_survived / num_event
p_survived
```

num_survived(1): 342

num_survived(2): 342

Out[5]: 0.3838383838383838

```
In [6]: # 더욱 컴팩트하게 해봅시다.  
# mean = sum / total_number 입니다.  
p_survived = df_titanic['Survived'].mean()  
p_survived
```

Out[6]: 0.3838383838383838

```
In [5]: Image('29.png')
```


Out[5]: **[Ex.2]**

1부터 6까지의 수가 적힌 모든 수가 동일한 확률로 나오는 정육면체 주사위 2번 던졌을 경우, 둘 중에 적어도 한 번 3의 배수가 나올 확률을 구해봅시다.

[풀이]

X_1 : 첫 번째 주사위의 수, X_2 : 두 번째 주사위의 수

- 방법-1

$$P(X_1 = 3\text{의 배수} \cup X_2 = 3\text{의 배수})$$

$$= P(X_1 = 3\text{의 배수}) + P(X_2 = 3\text{의 배수}) - P(X_1 = 3\text{의 배수} \cap X_2 = 3\text{의 배수})$$

$$= 1/3 + 1/3 - 1/9 = 5/9$$

- 방법2

$$P(X_1 = 3\text{의 배수} \cup X_2 = 3\text{의 배수}) = 1 - P(X_1 \neq 3\text{의 배수} \cap X_2 \neq 3\text{의 배수})$$

$$= 1 - 2/3 \times 2/3 = 5/9$$

[Ex.3]

$P(\text{Rain}) = 0.1$ 입니다, $P(\text{우산있음}|\text{Rain}) = 0.95$, $P(\text{우산있음}) = 0.3$ 일 때,

$P(\text{Rain}|\text{우산있음})$ 을 구해봅시다.

[풀이]

$$P(\text{Rain} | \text{우산 있음}) = \frac{P(\text{우산 있음} | \text{Rain}) \times P(\text{Rain})}{P(\text{우산 있음})} = \frac{0.95 \times 0.1}{0.3} = 0.31666\dots$$

In [6]: `Image('30.png')`Out[6]: **[Ex.4]**

df_titanic 에서 $P(\text{Survived} = \text{True} | \text{Pclass} = 1)$,
 $P(\text{Survived} = \text{True} | \text{Pclass} = 2)$,
 $P(\text{Survived} = \text{True} | \text{Pclass} = 3)$ 을 구하세요

In [7]: `df_titanic.groupby('Pclass')['Survived'].sum() / df_titanic['Pclass'].value_counts()`

Out[7]:

1	0.629630
2	0.472826
3	0.242363

dtype: float64

In [8]: `df_titanic.groupby('Pclass')['Survived'].mean()`

Out[8]:

Pclass	
1	0.629630
2	0.472826
3	0.242363

Name: Survived, dtype: float64

In [7]: `Image('31.png')`

Out[7]: **[Ex.5]**

df_titanic 에서 $P(\text{Survived} = \text{False} | \text{Pclass} = 1)$,
 $P(\text{Survived} = \text{False} | \text{Pclass} = 2)$,
 $P(\text{Survived} = \text{False} | \text{Pclass} = 3)$ 을 구하세요

In [9]: `1 - df_titanic.groupby('Pclass')['Survived'].mean()`

Out[9]: Pclass
 1 0.370370
 2 0.527174
 3 0.757637
 Name: Survived, dtype: float64

In [8]: `Image('32.png')`Out[8]: **[Ex.6]**

df_titanic 에서, $P(\text{Survived} = \text{True} | \text{Pclass}, \text{Sex}, \text{Embarked})$ 를

Pclass, Sex, Embarked 의 모든 표본 공간(사건 공간)에 대한 확률을 구해봅시다.

구하는 방법은,

Survived가 조건으로 주어진다면 Pclass, Sex 그리고 Embarked는 서로 독립임을 가정합니다.

Bayes Theorem을 이용합니다.

$$P(A_i | B) = \frac{P(B|A_i)P(A_i)}{\sum_j P(B|A_j)P(A_j)}$$

In [9]: `Image('33.png')`

Out[9]:
$$P(\text{Survived} = \text{True} | \text{Pclass}, \text{Sex}, \text{Embarked}) = \frac{P(\text{Pclass}, \text{Sex}, \text{Embarked} | \text{Survived} = \text{True}) P(\text{Survived} = \text{True})}{P(\text{Pclass}, \text{Sex}, \text{Embarked} | \text{Survived} = \text{True}) P(\text{Survived} = \text{True}) + P(\text{Pclass}, \text{Sex}, \text{Embarked} | \text{Survived} = \text{False}) P(\text{Survived} = \text{False})}$$

위 식을 정리해봅시다.

$$P(\text{Pclass}, \text{Sex}, \text{Embarked}, \text{Survived} = \text{True})$$

Chain Rule에 의해,

$$= P(\text{Pclass} | \text{Sex}, \text{Embarked}, \text{Survived} = \text{True}) P(\text{Sex} | \text{Embarked}, \text{Survived} = \text{True}) P(\text{Embarked} | \text{Survived} = \text{True}) P(\text{Survived} = \text{True})$$

Pclass와 Sex, Embarked가 Survived에 대하여 조건부 독립임을 이용하면,

$$= P(\text{Pclass} | \text{Survived} = \text{True}) P(\text{Sex} | \text{Survived} = \text{True}) P(\text{Embarked} | \text{Survived} = \text{True}) P(\text{Survived} = \text{True})$$

P(Survived=True)를 좌항으로 이항하면

$$P(\text{Pclass}, \text{Sex}, \text{Embarked} | \text{Survived} = \text{True})$$

$$= P(\text{Pclass} | \text{Survived} = \text{True}) P(\text{Sex} | \text{Survived} = \text{True}) P(\text{Embarked} | \text{Survived} = \text{True})$$

로 정리됩니다.

마찬가지로,

$$P(\text{Pclass}, \text{Sex}, \text{Embarked} | \text{Survived} = \text{False})$$

$$= P(\text{Pclass} | \text{Survived} = \text{False}) P(\text{Sex} | \text{Survived} = \text{False}) P(\text{Embarked} | \text{Survived} = \text{False})$$

가 됩니다.

```
In [10]: # Survived에 대한 조건부 확률을 구합니다.
df_pclass_prob = pd.crosstab(index=df_titanic['Pclass'],
                             columns=df_titanic['Survived'], normalize='columns')
df_sex_prob = pd.crosstab(index=df_titanic['Sex'],
                           columns=df_titanic['Survived'], normalize='columns')
df_embarked_prob = pd.crosstab(index=df_titanic['Embarked'],
                                columns=df_titanic['Survived'], normalize='columns')
```

```
In [11]: display(df_pclass_prob)
display(df_sex_prob)
display(df_embarked_prob)
```

Survived	0	1
Pclass		
1	0.145719	0.397661
2	0.176685	0.254386
3	0.677596	0.347953

Survived	0	1
Sex		
female	0.147541	0.681287
male	0.852459	0.318713

Survived	0	1
Embarked		
C	0.136612	0.271930
Q	0.085610	0.087719
S	0.777778	0.640351

```
In [12]: '''
코드는 `itertools.product()` 함수를 사용하여 세 개의 리스트에서
가능한 모든 조합을 생성하고, 이를 데이터프레임으로 변환하는 과정을 수행합니다.

1. `itertools.product()` 함수는 주어진 리스트들의 모든 요소들의
   조합을 생성합니다. 이 코드에서는 `df_pclass_prob.index`,
   `df_sex_prob.index`, `df_embarked_prob.index` 리스트들이 사용됩니다.
   각각은 데이터프레임의 인덱스를 나타냅니다.
```

2. 이 함수는 모든 가능한 조합을 튜플의 형태로 반환합니다.
이 튜플은 세 개의 요소를 가지며,
각 요소는 세 리스트의 요소들 중 하나입니다.
3. 리스트 컴프리헨션을 사용하여 모든 조합을 데이터프레임의
행으로 구성하는데, 각 행은 `(Pclass, Sex, Embarked)` 형태의 튜플입니다.
4. 이렇게 생성된 데이터프레임은 `df\_prob`에 할당됩니다.

결과적으로, `df\_prob` 데이터프레임은 `df\_pclass\_prob`, `df\_sex\_prob`,
`df\_embarked\_prob` 데이터프레임의 인덱스로부터 가능한
모든 조합을 포함하는 데이터프레임입니다.
'''

```
import itertools
# 가능한 itertools.product는 주어진 리스트들의
# 모든 경우의 수를 구해줍니다.
df_prob = pd.DataFrame(
    [(Pclass, Sex, Embarked) for Pclass, Sex, Embarked in itertools.product(
        df_pclass_prob.index, df_sex_prob.index, df_embarked_prob.index
    )],
    columns=['Pclass', 'Sex', 'Embarked']
)
df_prob
```

Out[12]:

	Pclass	Sex	Embarked
0	1	female	C
1	1	female	Q
2	1	female	S
3	1	male	C
4	1	male	Q
5	1	male	S
6	2	female	C
7	2	female	Q
8	2	female	S
9	2	male	C
10	2	male	Q
11	2	male	S
12	3	female	C
13	3	female	Q
14	3	female	S
15	3	male	C
16	3	male	Q
17	3	male	S

In []:

```
'''
이 코드는 주어진 데이터프레임 `x`와 생존 여부를 기반으로
각 승객의 생존 가능성(likelihood)을 계산하는 함수를 정의합니다.

1. `calc_likelihood` 함수는 두 개의 인자 `x`와
```

- ``survived``를 입력으로 받습니다.
 - ``x``는 승객 정보가 포함된 데이터프레임입니다.
이 데이터프레임은 적어도 `'Pclass'`, `'Sex'`, `'Embarked'` 열을 가져야 합니다.
 - ``survived``는 생존 여부를 나타내는 인덱스입니다.
2. 함수 내부에서는 ``x``의 각 열(`'Pclass'`, `'Sex'`, `'Embarked'`)을 각각 해당하는 생존 여부에 대한 확률로 매핑합니다.
 - ``x['Pclass'].map(df_pclass_prob[survived])``:
`'Pclass'` 열의 각 값에 대해 ``df_pclass_prob`` 데이터프레임에서 해당하는 생존 여부에 대한 확률을 가져와 매핑합니다.
 - ``x['Sex'].map(df_sex_prob[survived])``: `'Sex'` 열의 각 값에 대해 ``df_sex_prob`` 데이터프레임에서 해당하는 생존 여부에 대한 확률을 가져와 매핑합니다.
 - ``x['Embarked'].map(df_embarked_prob[survived])``:
`'Embarked'` 열의 각 값에 대해 ``df_embarked_prob`` 데이터프레임에서 해당하는 생존 여부에 대한 확률을 가져와 매핑합니다.
 3. 각 열에 대해 매핑된 확률들을 곱하여
각 승객의 생존 가능성을 계산합니다.
 4. 계산된 생존 가능성이 담긴 **Series**를 반환합니다.

이 함수를 사용하면 주어진 승객 정보를 바탕으로
해당 승객의 생존 가능성을 추정할 수 있습니다.

```
'''
```

```
def calc_liklihood(x, survived):
    """
    survived 조건에서 대해서 x에 해당하는 경우의
    우도(liklihood)를 계산합니다.
    Parameters
    x: DataFrame
        경우의 수를 나타내는 데이터프레임
    survived: int
        0: survived 1: not survived
    Returns
    float
        P(Pclass=x['Pclass']|Survived=survived)×
        P(Sex=x['Sex']|Survived=survived)×
        P(Pclass=x['Embarked']|Survived=survived)
    """
    return x['Pclass'].map(df_pclass_prob[survived]) * \
```



```
x['Sex'].map(df_sex_prob[survived]) * \
x['Embarked'].map(df_embarked_prob[survived])
```

In [13]:

'''

이 코드는 베이지안 통계의 개념을 사용하여
승객의 생존 가능성을 예측하기 위한 계산을 수행합니다.

1. `df_prob.assign()` 함수를 사용하여
새로운 열들을 데이터프레임에 추가합니다.
2. `prior=df_titanic['Survived'].mean()`은 'Survived' 열의 평균을 계산하여
사전 확률을 설정합니다. 이는 생존한 승객의 비율을 의미합니다.
3. `likelihood=lambda x: calc_likelihood(x, 1)`은
`calc_likelihood` 함수를 사용하여 승객이 생존한 경우의
우도(likelihood)를 계산합니다.
이는 생존 여부에 대한 조건부 확률로,
'1'을 인자로 전달하여 생존한 경우의 확률을 계산합니다.
4. `evidence=lambda x:calc_likelihood(x, 0) *
(1 - df_titanic['Survived'].mean()) +
calc_likelihood(x, 1) *`
`df_titanic['Survived'].mean()`은 베이즈 정리를 사용하여
증거(evidence)를 계산합니다.
여기서 '0'은 생존하지 않은 경우를 나타냅니다.
전체 증거는 생존한 경우와 생존하지 않은 경우에 대한
우도와 각각의 확률을 고려하여 계산됩니다.
5. `posterior=lambda x: x['likelihood'] * x['prior'] / x['evidence']`는
베이지안 업데이트를 통해 사후 확률을 계산합니다.
사후 확률은 사전 확률, 우도 및 증거를 고려하여 계산됩니다.

결과적으로, 이 코드는 각 승객에 대한 사전 확률, 우도, 증거,
그리고 사후 확률을 계산하여 데이터프레임에 추가합니다.
이를 통해 각 승객의 생존 가능성을 예측할 수 있습니다.

'''

```
df_prob.assign(
    # 사전확률을 계산합니다.
    prior=df_titanic['Survived'].mean(),
    # 우도(likelihood)를 계산합니다.
    likelihood=lambda x: calc_likelihood(x, 1),
    evidence=lambda x:calc_likelihood(x, 0) * (1 - df_titanic['Survived'].mean()) + \
    # 전확률 법칙을 이용한 증거를 계산합니다.
```

```

        calc_liklihood(x, 1) * df_titanic['Survived'].mean()
    ).assign(
        posterior=lambda x: x['liklihood'] * x['prior'] / x['evidence']
    )

```

Out[13]:

	Pclass	Sex	Embarked	prior	liklihood	evidence	posterior
0	1	female	C	0.383838	0.073671	0.030088	0.939851
1	1	female	Q	0.383838	0.023765	0.010256	0.889421
2	1	female	S	0.383838	0.173484	0.076893	0.866004
3	1	male	C	0.383838	0.034464	0.023685	0.558529
4	1	male	Q	0.383838	0.011118	0.010820	0.394397
5	1	male	S	0.383838	0.081158	0.090682	0.343524
6	2	female	C	0.383838	0.047128	0.020284	0.891821
7	2	female	Q	0.383838	0.015203	0.007210	0.809291
8	2	female	S	0.383838	0.110979	0.055091	0.773232
9	2	male	C	0.383838	0.022047	0.021141	0.400295
10	2	male	Q	0.383838	0.007112	0.010675	0.255727
11	2	male	S	0.383838	0.051917	0.092109	0.216351
12	3	female	C	0.383838	0.064463	0.033158	0.746211
13	3	female	Q	0.383838	0.020794	0.013255	0.602153
14	3	female	S	0.383838	0.151799	0.106177	0.548765
15	3	male	C	0.383838	0.030156	0.060197	0.192289
16	3	male	Q	0.383838	0.009728	0.034203	0.109168
17	3	male	S	0.383838	0.071013	0.304076	0.089641

In []: